

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Andriy Shevtiyan - 5024231080

2026

1 Langkah-Langkah Percobaan

Praktikum Modul 1 dilaksanakan pada 8 November 2026. Seluruh percobaan memakai satu project PlatformIO bernama `modul1-praktikum` di Visual Studio Code. Folder project diletakkan pada path tanpa spasi. Setiap percobaan hanya mengganti isi `platformio.ini` dan file kode di `src/`. Alat dan bahan mengikuti Modul 1: STM32 Blackpill, ST-Link, ESP32 DevKit, pushbutton, resistor 10 kΩ, serta kabel jumper.

1.1 Percobaan 1: STM32 Blackpill (Blink)

1. Mengisi `platformio.ini` dengan konfigurasi berikut, lalu menunggu PlatformIO selesai me-refresh konfigurasi.

Listing 1: platformio.ini Percobaan 1

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Menghubungkan ST-Link ke Blackpill sesuai Tabel 1, lalu menghubungkan ST-Link ke komputer melalui USB.

Tabel 1: Sambungan ST-Link ke STM32 Blackpill

Pin ST-Link	Pin Blackpill	Fungsi
SWDIO	SWDIO	Jalur data debug
SWCLK	SWCLK	Jalur clock debug
GND	GND	Ground bersama
3.3V	3V3	Tegangan target

3. Menulis program Blink pada `src/main.cpp`, lalu menjalankan Build dan Upload.

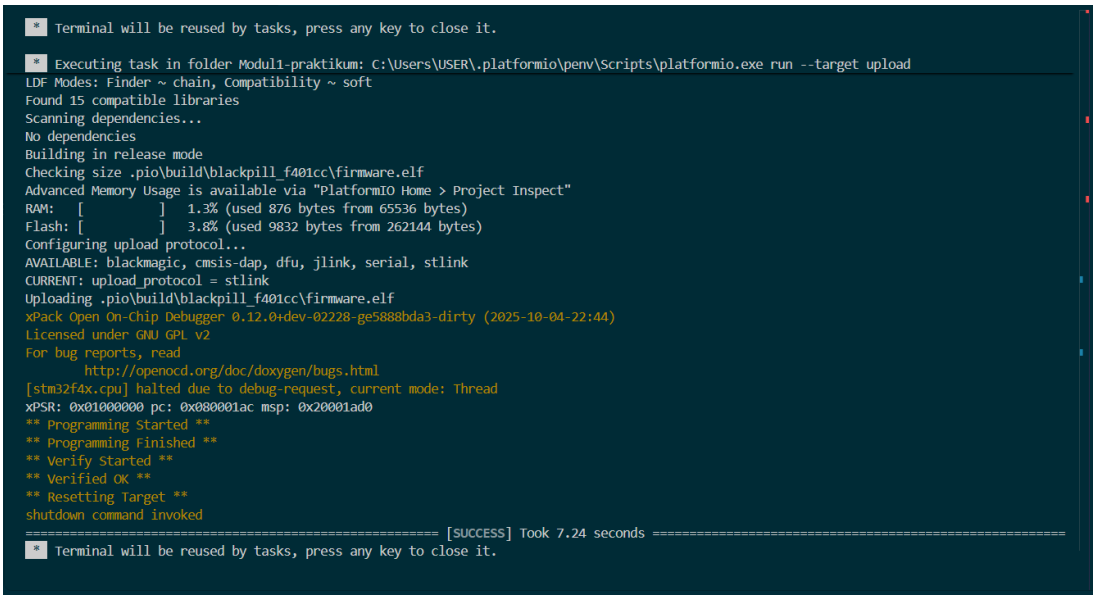
Listing 2: Program Blink STM32 (Arduino)

```
#include <Arduino.h>

#define LED_PIN PC13

void setup()
{
  pinMode(LED_PIN, OUTPUT);
}

void loop()
{
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW
  delay(500);
  digitalWrite(LED_PIN, HIGH);
  delay(500);
}
```



Gambar 1: Hasil Build dan Upload Percobaan 1 pada terminal PlatformIO

4. Mengamati LED onboard pada pin PC13.

Upload pertama sempat gagal. Penyebab dan penanganannya dibahas pada bagian Analisis Hasil Percobaan.

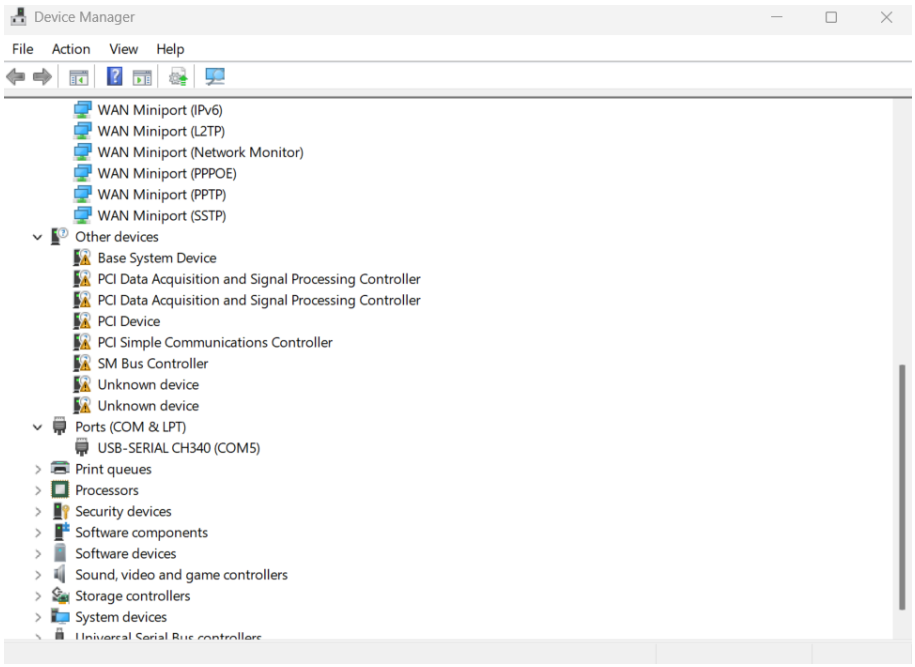
1.2 Percobaan 2: ESP32 dengan Framework ESP-IDF (Blink)

1. Mengganti isi `platformio.ini` dengan konfigurasi ESP-IDF.

Listing 3: platformio.ini Percobaan 2

```
1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = espidf
```

2. Menghubungkan ESP32 ke komputer melalui USB dan memastikan port serialnya terdeteksi (pada kami COM5).



Gambar 2: Device Manager: ESP32 terdeteksi sebagai USB-SERIAL CH340 (COM5)

3. Menghapus `src/main.cpp`, membuat `src/main.c`, menulis program berikut, lalu menjalankan Build dan Upload.

Listing 4: Program Blink ESP32 (ESP-IDF)

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

4. Mengamati LED onboard ESP32 pada GPIO 2.

1.3 Percobaan 3: Pull-up dan Pull-down, Eksternal dan Internal

Bagian A: resistor eksternal.

1. Mengembalikan `platformio.ini` ke framework Arduino, menghapus `src/main.c`, dan membuat kembali `src/main.cpp`. Pada percobaan ini kami menambahkan `monitor_speed`

= 115200 agar sama dengan `Serial.begin(115200)` pada kode.

Listing 5: platformio.ini Percobaan 3

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
```

2. Merangkai dua pushbutton: tombol pull-up pada GPIO 32 (3V3 melalui resistor 10 kΩ ke node GPIO 32, tombol dari node ke GND) dan tombol pull-down pada GPIO 33 (GND melalui resistor 10 kΩ ke node GPIO 33, tombol dari node ke 3V3).

3. Menulis Kode Program A, menjalankan Build dan Upload, membuka serial monitor, lalu menekan kedua tombol bergantian.

Listing 6: Kode Program A: pull-up dan pull-down eksternal

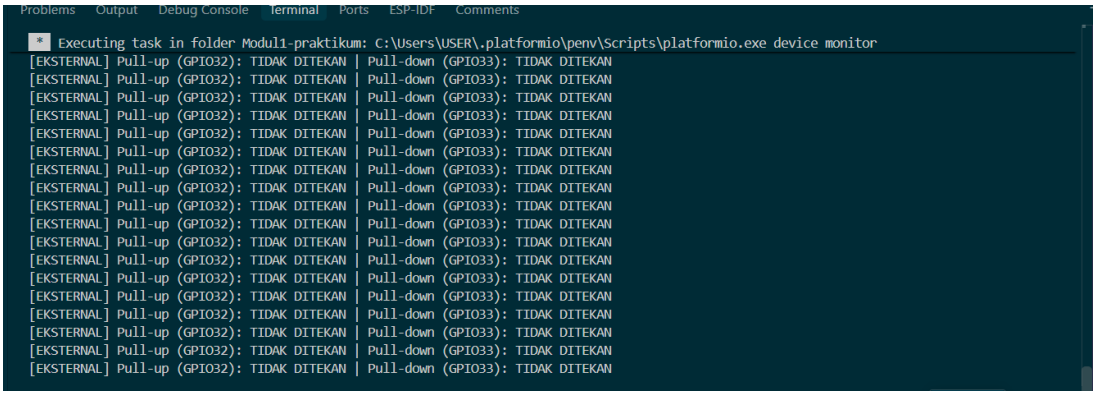
```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32    // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33  // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33):
      %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```



Gambar 3: Keluaran serial monitor Kode Program A setelah wiring diperbaiki (tombol tidak ditekan)

Bagian B: resistor internal.

1. Melepas kedua resistor eksternal, tombol tetap terpasang pada GPIO 32 (kaki lain ke GND) dan GPIO 33 (kaki lain ke 3V3).

2. Mengganti isi `src/main.cpp` dengan Kode Program B, menjalankan Build dan Upload, lalu menekan kedua tombol.

Listing 7: Kode Program B: pull-up dan pull-down internal

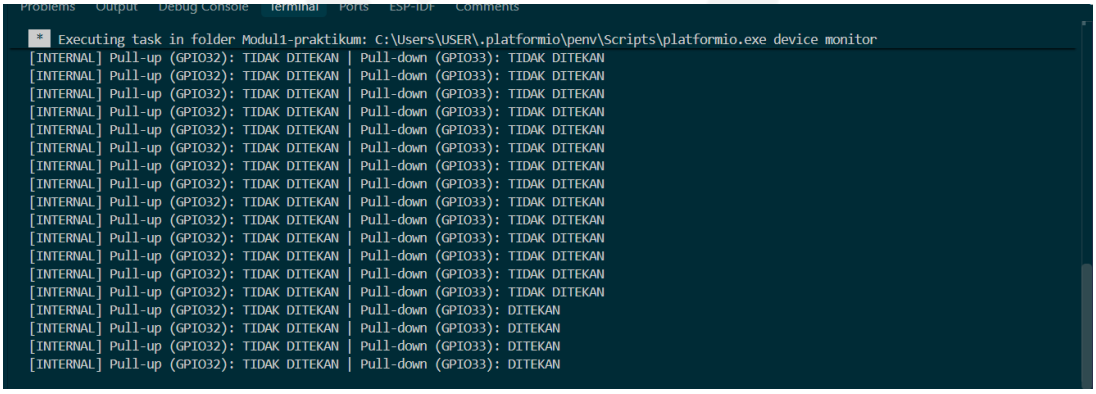
```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32    // pull-up internal
#define BUTTON_PULLDOWN_PIN 33  // pull-down internal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %
      s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```



Gambar 4: Keluaran serial monitor Kode Program B: tombol tidak ditekan, kemudian tombol pull-down (GPIO 33) ditekan

1.4 Percobaan 4 dan 5

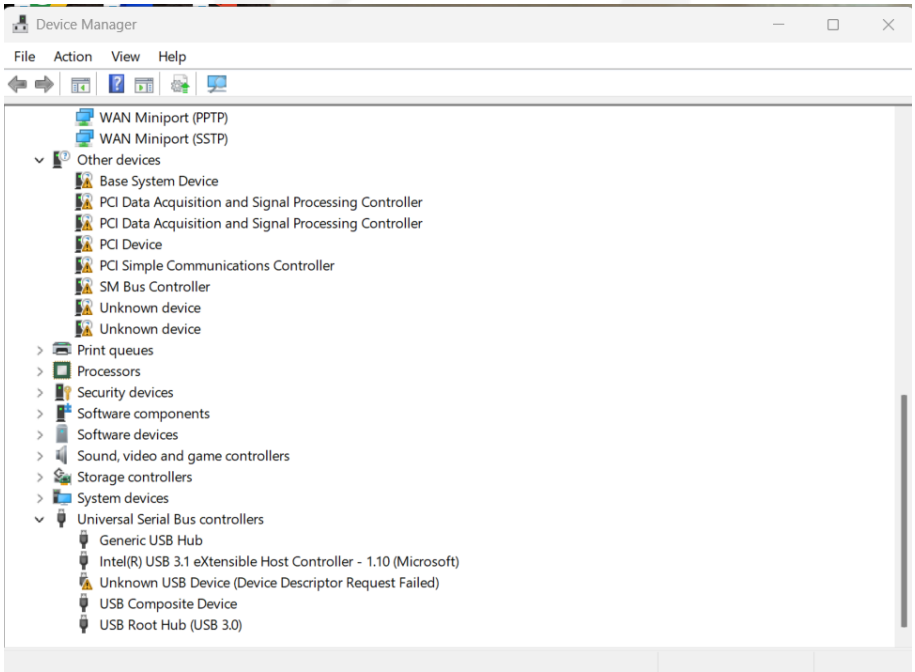
Percobaan 4 (debouncing) dan Percobaan 5 (ADC dengan LDR) tidak sempat dilaksanakan karena waktu praktikum habis setelah Percobaan 3. Oleh karena itu laporan ini tidak memuat langkah, data, maupun analisis hasil untuk kedua percobaan tersebut.

2 Analisis Hasil Percobaan

2.1 Percobaan 1

Hasil pengamatan menunjukkan LED onboard pada pin PC13 berkedip secara teratur: menyala selama 500 ms kemudian padam selama 500 ms. Satu siklus penuh berlangsung 1000 ms, artinya frekuensi kedipan 1 Hz dengan duty cycle 50%. Nilai ini cocok dengan isi program, karena di dalam satu putaran `loop()` terdapat dua kali pemanggilan `delay(500)`. Perlu diperhatikan bahwa LED onboard Blackpill bekerja dengan logika aktif LOW: anoda LED tersambung ke 3,3 V sedangkan katodanya ke pin PC13, sehingga arus baru mengalir dan LED menyala saat pin bernilai LOW. Inilah alasan perintah `digitalWrite(LED_PIN, LOW)` justru menyalakan LED, sedangkan `HIGH` memadamkannya, sejalan dengan penjelasan pada modul.

Pada percobaan upload yang pertama, proses gagal dengan pesan *OpenOCD Error: open failed*, dan pada Device Manager muncul perangkat *Unknown USB Device (Device Descriptor Request Failed)*. Karena tahap build sudah selesai tanpa error, sumber masalah dapat dipastikan bukan pada program, melainkan pada jalur komunikasi antara komputer dan ST-Link lewat USB. Langkah pertama yang kami ambil adalah memasang driver ST-Link (paket `stsw-link009`), namun tanda seru pada Device Manager belum hilang. Masalah baru teratasi setelah sambungan kabel SWD dicek ulang dan kesalahan pemasangan kabel dibetulkan, sehingga upload dapat berjalan. Dari kejadian ini dapat diambil pola penelusuran masalah yang runtut: periksa dahulu kode (apakah build berhasil), kemudian driver, dan terakhir koneksi fisik.



Gambar 5: Device Manager menampilkan *Unknown USB Device (Device Descriptor Request Failed)* saat upload pertama gagal

2.2 Percobaan 2

LED onboard ESP32 yang terhubung ke GPIO 2 berkedip dengan periode 1000 ms, identik dengan Percobaan 1. Hal ini membuktikan bahwa program berbasis ESP-IDF sudah berhasil dijalankan pada ESP32. Karena fungsi kedua program sama-sama Blink, perbedaan antara framework Arduino dan ESP-IDF lebih tampak dari cara program disusun, sebagaimana dirangkum pada Tabel 2.

Tabel 2: Perbandingan program Blink dengan Arduino dan ESP-IDF

Aspek	**Arduino**	**ESP-IDF**
Titik masuk	`setup()` sekali, `loop()` berulang	`app_main()`, perulangan ditulis sendiri
Konfigurasi pin	`pinMode()`	`gpio_reset_pin()` dan `gpio_set_direction()`
Menulis level pin	`digitalWrite()`	`gpio_set_level()`
Penundaan	`delay()` (ms)	`vTaskDelay(pdMS_TO_TICKS())`
Berkas utama	`main.cpp`	`main.c`

Program ESP-IDF memerlukan jumlah baris yang lebih banyak serta pemahaman terhadap API tingkat rendah, tetapi sebagai gantinya pengembang dapat memanfaatkan FreeRTOS dan fitur-fitur ESP32 secara langsung. Sebaliknya, Arduino terasa lebih sederhana karena framework menyembunyikan detail-detail tersebut dari pengguna. Perlu dicatat bahwa framework Arduino untuk ESP32 sebenarnya juga dibangun di atas FreeRTOS. Jadi, yang membedakan keduanya adalah tingkat abstraksi dan seberapa besar kendali yang diberikan kepada pengembang, bukan keberadaan RTOS itu sendiri.

Terdapat dua kendala teknis yang menentukan berhasil tidaknya build ESP-IDF di PlatformIO. Kendala pertama muncul ketika persiapan sebelum praktikum: project yang disimpan pada path yang mengandung spasi tidak dapat dikonfigurasi dan memunculkan pesan *Detected a whitespace character in project paths*. Solusinya adalah memindahkan project ke path tanpa spasi. Kendala kedua berkaitan dengan nama file: program ESP-IDF perlu disimpan dengan nama `main.c`. Jika ditulis dalam file C++, nama fungsi akan mengalami *name mangling* oleh compiler sehingga fungsi `app_main` tidak ditemukan, kecuali fungsi tersebut dideklarasikan dengan `extern "C"`.

2.3 Percobaan 3

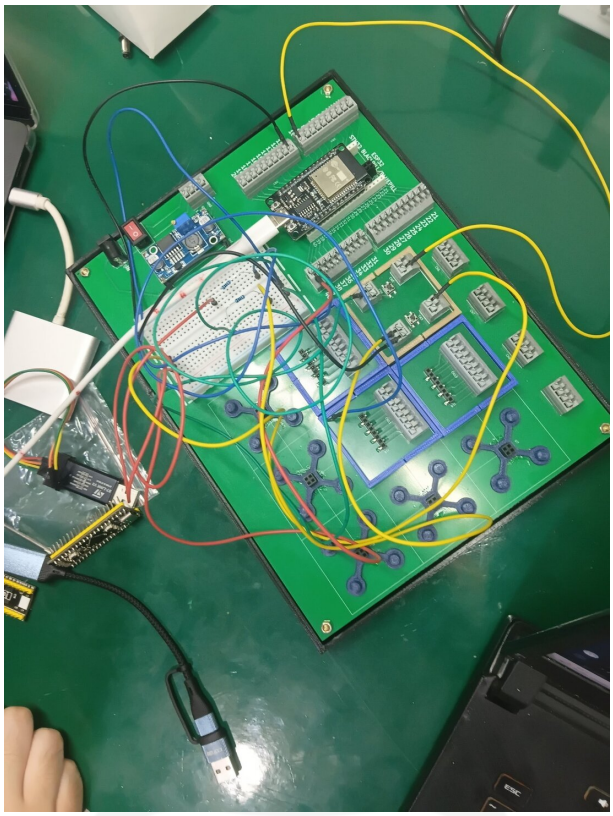
Perilaku pin yang seharusnya terjadi berdasarkan teori ditunjukkan pada Tabel 3. Program kemudian menerjemahkan level logika tersebut menjadi teks DITEKAN atau TIDAK DITEKAN pada serial monitor.

Tabel 3: Level pin yang diharapkan pada Percobaan 3

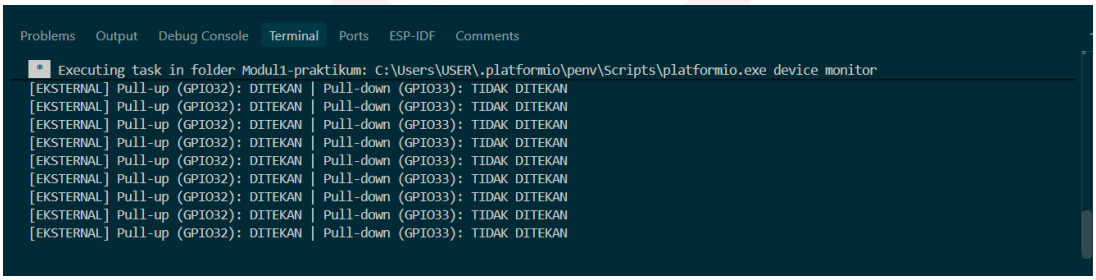
Kondisi tombol	**Pull-up (GPIO 32)**	**Pull-down (GPIO 33)**
Tidak ditekan	HIGH (TIDAK DITEKAN)	LOW (TIDAK DITEKAN)
Ditekan	LOW (DITEKAN)	HIGH (DITEKAN)

Pada rangkaian pull-up, ketika tombol tidak ditekan resistor menahan pin pada tegangan 3,3 V, dan ketika tombol ditekan pin terhubung langsung ke GND; dengan demikian rangkaian ini berlogika aktif LOW. Rangkaian pull-down bekerja kebalikannya: resistor menahan pin pada GND saat tombol terbuka, lalu tombol menyambungkan pin ke 3,3 V saat ditekan, sehingga logikanya aktif HIGH. Perbedaan logika inilah yang diakomodasi dalam program melalui kondisi `stateUp == LOW` dan `stateDown == HIGH`.

Saat pengujian pertama, serial monitor terus menampilkan status DITEKAN untuk GPIO 32 (pull-up) meskipun tombol sama sekali tidak disentuh. Artinya, pin tersebut selalu terbaca LOW. Kondisi ini sesuai dengan dugaan bahwa pin tidak ditarik ke 3,3 V dan/atau terhubung ke GND karena pemasangan kabel yang keliru, sehingga rangkaian tidak bekerja sebagai pull-up. Wiring kemudian dibetulkan oleh asisten praktikum tanpa penjelasan detail, dan kami tidak sempat mencatat bagian yang diubah, sehingga penyebab pastinya tidak dapat dipastikan. Sesudah perbaikan, GPIO 32 (pull-up) maupun GPIO 33 (pull-down) terbaca TIDAK DITEKAN ketika kedua tombol dibiarkan, sesuai dengan Tabel 3 (lihat Gambar 3). Namun, pembacaan saat tombol ditekan pada Bagian A tidak terekam dengan bukti yang jelas, sehingga kondisi DITEKAN yang benar belum dapat dibuktikan melalui tangkapan layar.



Gambar 6: Rangkaian pada trainer board saat GPIO 32 terbaca DITEKAN terus (sebelum wiring diperbaiki)



Gambar 7: Serial monitor saat GPIO 32 (pull-up) terbaca DITEKAN terus walaupun tombol tidak ditekan

Fungsi utama resistor pull-up dan pull-down adalah mencegah pin input berada dalam keadaan *floating*. Pin yang mengambang memiliki impedansi masukan yang sangat besar, sehingga level logikanya tidak pasti dan mudah terpengaruh noise dari lingkungan sekitar. Kasus pada pengujian awal memperlihatkan bahwa jalur pull-up yang salah tidak hanya menimbulkan pembacaan acak, tetapi juga dapat menghasilkan pembacaan keliru yang konsisten.

Secara fungsi, resistor eksternal 10 kΩ dan resistor internal tidak berbeda, tetapi karakteristiknya dalam praktik berbeda. Pada pull-up 10 kΩ dengan catu 3,3 V, arus yang mengalir ke GND saat tombol ditekan sebesar $3{,}3/10.000 = 0{,}33$ mA. Resistor pull internal ESP32 memiliki nilai tipikal sekitar 45 kΩ menurut datasheet, sehingga arusnya hanya sekitar 0,07 mA. Keunggulan resistor internal adalah tidak memerlukan komponen dan kabel tambahan, namun tarikannya lebih lemah sehingga lebih mudah terganggu noise terutama bila kabel panjang.

Resistor eksternal memberikan tarikan yang lebih kuat dan nilainya bebas dipilih sesuai kebutuhan. Pada Kode Program B (dengan pin dan tombol yang sama), serial monitor menunjukkan GPIO 32 (pull-up internal) dan GPIO 33 (pull-down internal) sama-sama TIDAK DITEKAN ketika tombol dibiarkan, dan GPIO 33 berganti menjadi DITEKAN ketika tombol pull-down ditekan (Gambar 4). Pola tersebut identik dengan Kode Program A dan membuktikan bahwa mode `INPUT_PULLUP` serta `INPUT_PULLDOWN` mengaktifkan resistor internal yang berperan sama seperti resistor eksternal, sehingga rangkaian tetap berfungsi meskipun resistor 10 kΩ telah dilepas. Pembacaan GPIO 32 (pull-up) saat tombol ditekan tidak tertangkap pada tangkapan layar yang tersedia, sehingga kondisi tersebut tidak dapat didokumentasikan.

2.4 Percobaan 4 dan 5

Karena kedua percobaan ini tidak dikerjakan, tidak ada data yang dapat dianalisis. Pengaruh variasi waktu debounce (10, 50, dan 200 ms) serta pembacaan ADC menggunakan LDR belum diuji secara langsung. Konsep debouncing dan ADC hanya diterapkan pada Tugas Modul, dan itu pun masih dalam bentuk simulasi.

3 Hasil Tugas Modul

3.1 Rancangan

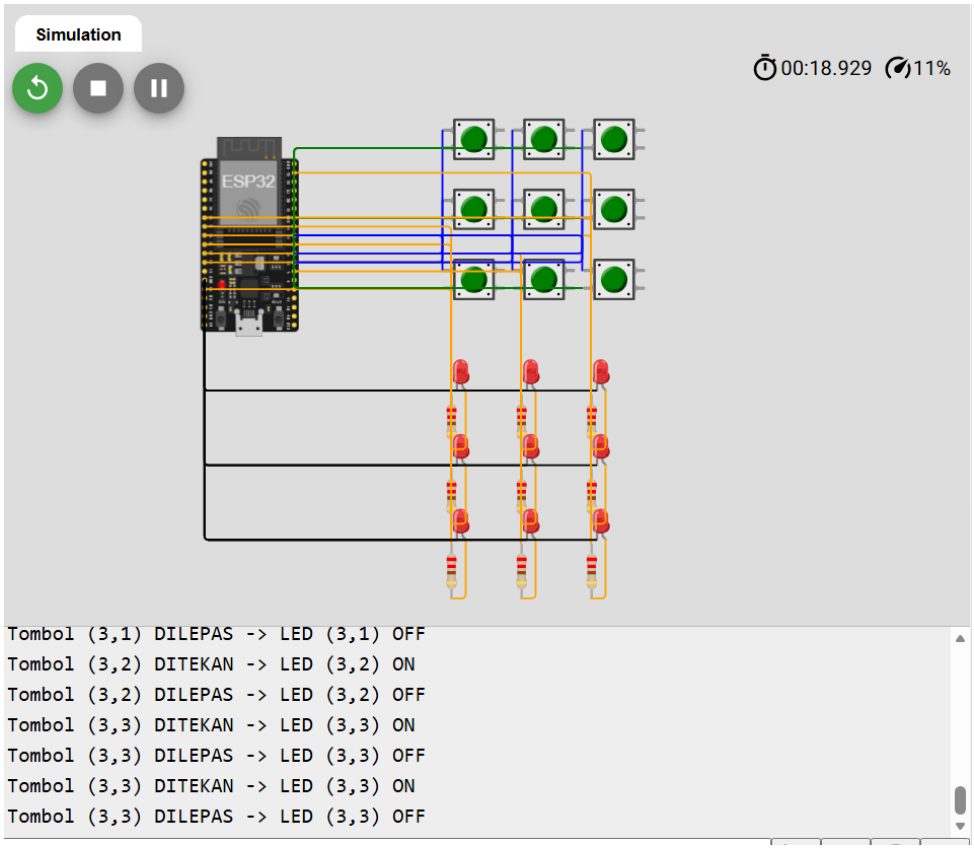
Tugas modul ini adalah membuat simulasi matriks pushbutton 3x3 untuk mengendalikan matriks LED 3x3. Setiap tombol pada posisi (baris, kolom) harus menyalakan LED pada posisi yang sama, dan program wajib dilengkapi debouncing. Simulasi dibangun di Wokwi menggunakan ESP32 DevKit. Pembacaan matriks tombol menggunakan metode *scanning*, dengan pin baris berperan sebagai output dan pin kolom sebagai input yang diberi pull-up internal. Matriks LED tersusun atas sembilan LED, masing-masing dikendalikan oleh satu pin GPIO dan dilengkapi resistor 220 Ω, serta diatur dengan tata letak yang sama seperti tombol. Pembagian pin selengkapnya tercantum pada Tabel 4.

Tabel 4: Pemetaan pin matriks 3x3

Fungsi	**Baris/Kolom 1**	**Baris/Kolom 2**	**Baris/Kolom 3**
Baris tombol (output)	GPIO 16	GPIO 17	GPIO 18
Kolom tombol (input pull-up)	GPIO 19	GPIO 21	GPIO 22
LED baris 1	GPIO 13	GPIO 14	GPIO 25
LED baris 2	GPIO 26	GPIO 27	GPIO 32
LED baris 3	GPIO 33	GPIO 4	GPIO 23

Dalam setiap siklus scanning, hanya satu baris yang diberi level LOW, sedangkan baris lainnya diatur ke kondisi high-Z (mode input). Apabila ada tombol yang ditekan pada baris yang sedang aktif, pin kolom yang bersangkutan akan terbaca LOW. Debouncing diterapkan

untuk masing-masing tombol secara terpisah: setiap pembacaan mentah dibandingkan dengan pembacaan sebelumnya, dan bila terjadi perubahan maka pencatat waktu diatur ulang. Status tombol baru dianggap sah, dan LED baru diperbarui, setelah pembacaan tidak berubah selama `DEBOUNCE_MS` = 30 ms. Dengan cara ini LED mengikuti status tombol yang telah stabil, yaitu menyala selama tombol masih ditekan.



Gambar 8: Rangkaian simulasi Wokwi saat tidak ada tombol ditekan: ESP32, matriks pushbutton 3x3, dan matriks LED 3x3 (seluruh LED mati)

3.2 Kode Program

Listing 8: Kode program matriks pushbutton 3x3 dengan debouncing

```
#include <Arduino.h>

// ===== Matrix Pushbutton 3x3 -> Matrix LED 3x3 (ESP32, framework Arduino) =====
// Baris tombol : OUTPUT (diaktifkan LOW satu per satu saat scanning)
// Kolom tombol : INPUT_PULLUP (pull-up internal, tanpa resistor eksternal)
// LED          : satu pin per LED (resistor 220 ohm), posisi sama dengan tombol

const uint8_t ROW_PINS[3] = {16, 17, 18};
const uint8_t COL_PINS[3] = {19, 21, 22};
const uint8_t LED_PINS[3][3] = {
    {13, 14, 25},
    {26, 27, 32},
    {33, 4, 23}};

```

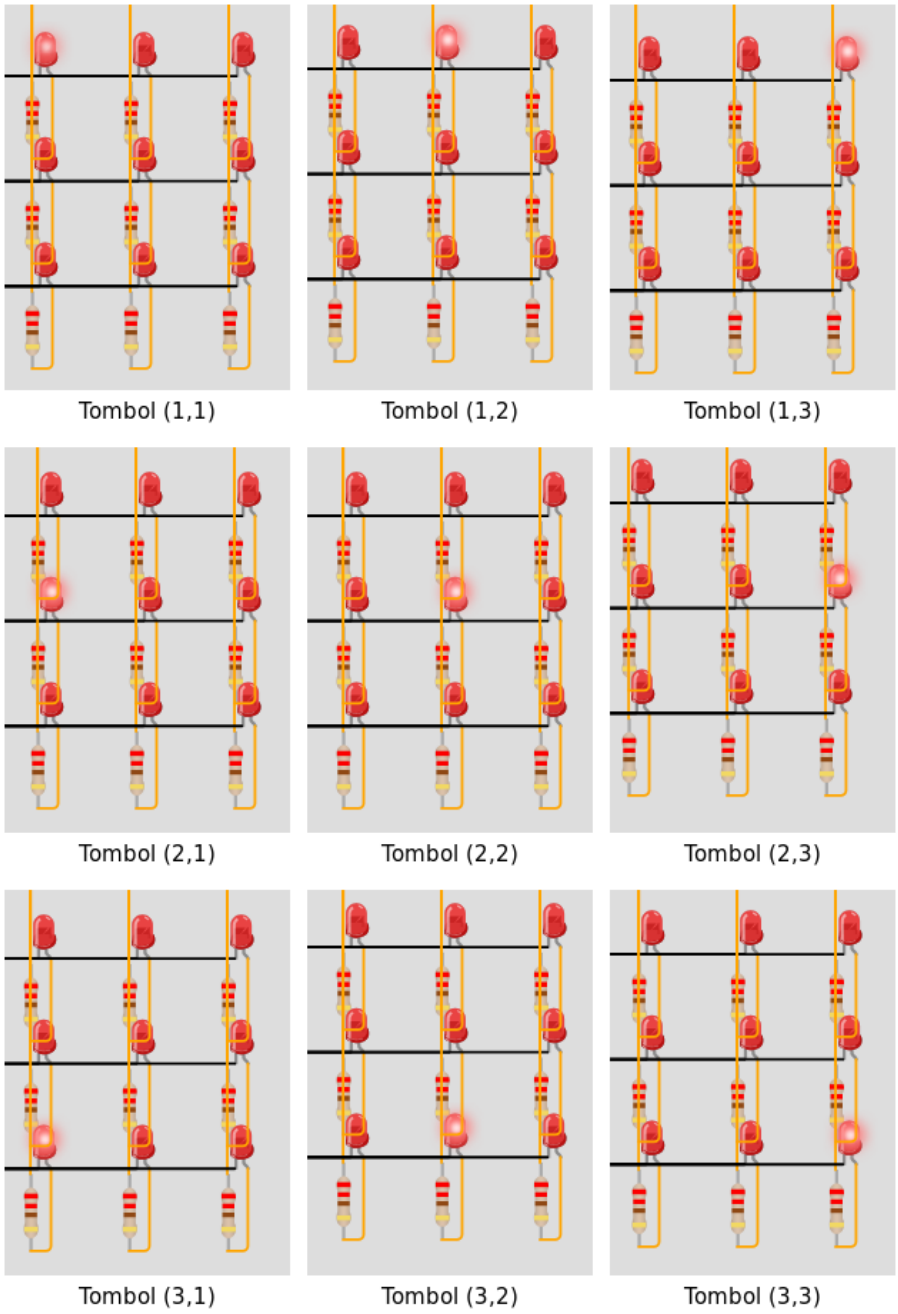
```
15 const uint32_t DEBOUNCE_MS = 30; // waktu minimum sinyal harus stabil
16
17 bool rawState[3][3];         // pembacaan mentah terakhir (true = ditekan)
18 bool stableState[3][3];      // status stabil, sudah lolos debouncing
19 uint32_t lastChange[3][3];   // waktu terakhir pembacaan mentah berubah
20
21 void setup()
22 {
23   Serial.begin(115200);
24
25   for (int r = 0; r < 3; r++)
26   {
27     pinMode(ROW_PINS[r], INPUT); // baris dibiarkan high-Z sampai giliran di-
         scan
28   }
29   for (int c = 0; c < 3; c++)
30   {
31     pinMode(COL_PINS[c], INPUT_PULLUP); // tidak ditekan = HIGH, ditekan =
         LOW
32   }
33   for (int r = 0; r < 3; r++)
34   {
35     for (int c = 0; c < 3; c++)
36     {
37       pinMode(LED_PINS[r][c], OUTPUT);
38       digitalWrite(LED_PINS[r][c], LOW);
39       rawState[r][c] = false;
40       stableState[r][c] = false;
41       lastChange[r][c] = 0;
42     }
43   }
44   Serial.println("Matrix pushbutton 3x3 siap");
45 }
46
47 void loop()
48 {
49   uint32_t now = millis();
50
51   for (int r = 0; r < 3; r++)
52   {
53     // Aktifkan baris r: tarik ke LOW, baris lain tetap high-Z
54     pinMode(ROW_PINS[r], OUTPUT);
55     digitalWrite(ROW_PINS[r], LOW);
56     delayMicroseconds(10); // beri waktu sinyal settle sebelum dibaca
57
58     for (int c = 0; c < 3; c++)
59     {
60       bool raw = (digitalRead(COL_PINS[c]) == LOW); // LOW = kontak tertutup
61
62       // Pembacaan mentah berubah -> mulai hitung ulang waktu stabil
63       if (raw != rawState[r][c])
64       {
65         rawState[r][c] = raw;
66         lastChange[r][c] = now;
67       }
68
69       // Sudah stabil minimal DEBOUNCE_MS dan berbeda dari status sah ->
           terima
70       if ((now - lastChange[r][c]) >= DEBOUNCE_MS && stableState[r][c] !=
           rawState[r][c])
71       {
72         stableState[r][c] = rawState[r][c];
73         digitalWrite(LED_PINS[r][c], stableState[r][c] ? HIGH : LOW);
```

```
74          Serial.printf("Tombol (%d,%d) %s -> LED (%d,%d) %s\n",
75                        r + 1, c + 1, stableState[r][c] ? "DITEKAN" : "DILEPAS"
                              ,
76                        r + 1, c + 1, stableState[r][c] ? "ON" : "OFF");
77        }
78      }
79
80      pinMode(ROW_PINS[r], INPUT); // lepas baris r sebelum scan baris
            berikutnya
81    }
82  }
```

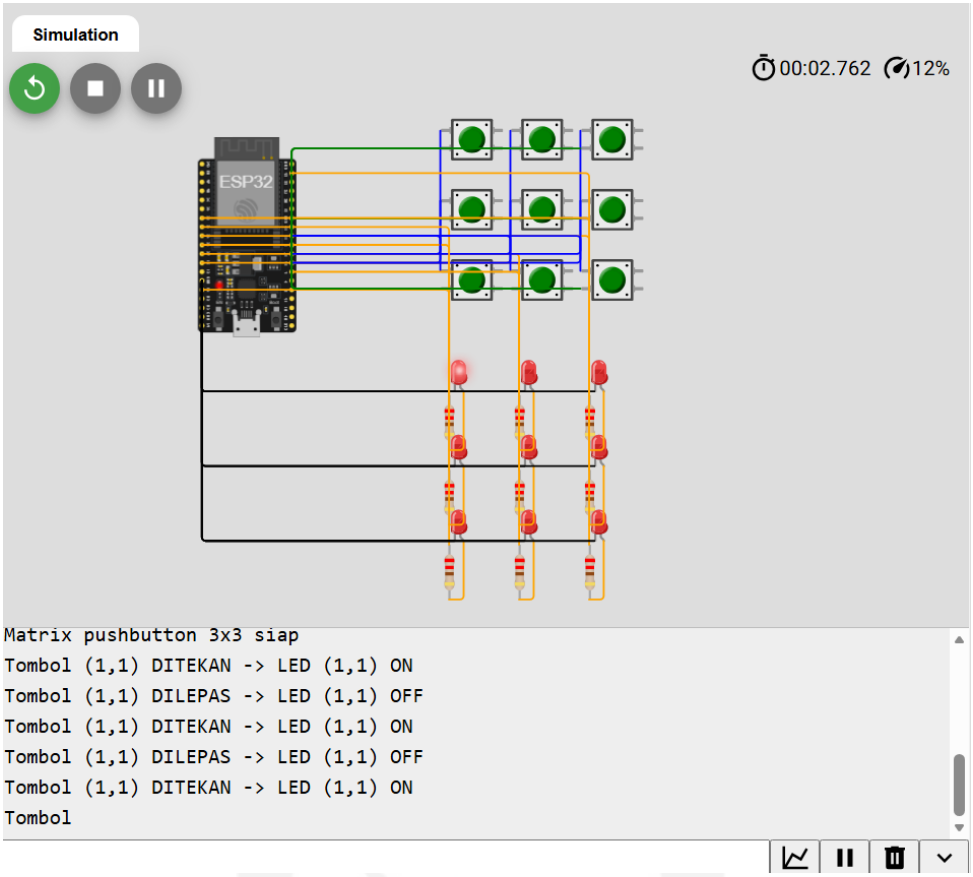
3.3 Hasil Simulasi

Pengujian dilakukan pada simulator Wokwi dengan menekan lalu melepas kesembilan tombol secara bergiliran. Setiap kali sebuah tombol ditekan, LED yang posisinya (baris, kolom) sama dengan tombol tersebut menyala dan tetap menyala hingga tombol dilepas, sementara delapan LED yang lain tetap padam. Kesembilan pasangan tombol–LED berperilaku sesuai pemetaan pada Tabel 4. Serial monitor juga merekam setiap perubahan status dengan benar: muncul satu baris DITEKAN ketika tombol ditekan dan satu baris DILEPAS ketika tombol dilepas, tanpa ada baris yang tercetak ganda untuk sekali penekanan.

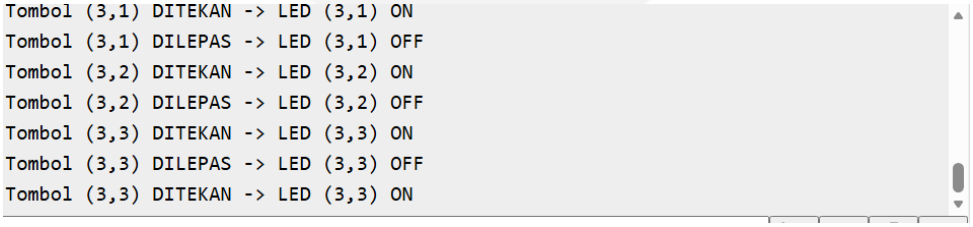
Pengujian ini memiliki beberapa keterbatasan. Tombol hanya diuji satu per satu, sehingga kondisi ketika beberapa tombol ditekan bersamaan belum diperiksa. Di samping itu, komponen pushbutton pada simulator tidak mensimulasikan pantulan kontak mekanik, sehingga efek debouncing tidak dapat diamati secara langsung pada simulasi; keberhasilannya hanya dapat dinilai dari logika program dan penjelasan pada bagian jawaban analisis.



Gambar 9: Matriks LED pada simulasi untuk kesembilan tombol yang ditekan satu per satu: hanya LED dengan posisi yang sama yang menyala



Gambar 10: Simulasi dengan tombol baris 1 kolom 1 ditekan: LED baris 1 kolom 1 menyala dan serial monitor mencatat kejadian



Gambar 11: Keluaran serial monitor saat tombol ditekan dan dilepas

3.4 Jawaban Pertanyaan Analisis

1. **Mengapa debouncing tetap diperlukan?** Bouncing merupakan karakteristik fisik yang dimiliki setiap kontak mekanik dan tidak bergantung pada berapa banyak tombol yang digunakan. Ketika kontak menutup atau membuka, bilah logam di dalam tombol memantul beberapa kali dalam rentang waktu yang sangat singkat (dalam orde milidetik), sehingga sinyal berganti-ganti antara HIGH dan LOW sebelum akhirnya stabil, sesuai dasar teori debouncing. Susunan matriks 3x3 hanya mengubah cara tombol dibaca, sedangkan masing-masing tombol tetap merupakan kontak mekanik yang dapat memantul. Karena loop scanning membaca tombol jauh lebih cepat dibandingkan lamanya pantulan, satu kali tekan dapat terdeteksi sebagai beberapa kali perubahan. Tanpa debouncing, LED

akan berkedip atau berganti status mengikuti pantulan tersebut, dan pada program yang menghitung jumlah penekanan, satu tekanan dapat terhitung lebih dari sekali. Dengan mensyaratkan sinyal stabil selama 30 ms, pantulan dapat diabaikan sehingga LED hanya berubah sekali untuk setiap penekanan. Gangguan sesaat (glitch) yang berlangsung kurang dari 30 ms juga otomatis tersaring.

2. **Konfigurasi pull-up pada pembacaan kolom.** Pin kolom diatur dalam mode `INPUT_PULLUP`, sehingga resistor internal menarik pin tersebut ke 3,3 V. Ketika tombol tidak ditekan, kolom terbaca HIGH; ketika tombol ditekan dan barisnya sedang aktif (LOW), kolom terhubung ke baris dan terbaca LOW, sehingga logikanya aktif LOW. Keadaan HIGH ini harus dijamin oleh pull-up dan tidak boleh dibiarkan floating, karena input CMOS memiliki impedansi yang sangat tinggi sehingga level tegangannya menjadi tidak pasti dan mudah berubah oleh noise, induksi dari kabel, maupun muatan liar. Pembacaan yang tidak stabil akan membuat tombol seolah-olah ditekan padahal tidak, dan LED pun berkedip dengan sendirinya. Gejala serupa kami alami pada Percobaan 3, ketika jalur pull-up yang tidak terpasang dengan benar menyebabkan GPIO 32 selalu terbaca LOW dan tombol dianggap terus ditekan.

3. **Digital input dan analog input (ADC).** Pada digital input, mikrokontroler hanya membandingkan tegangan pada pin terhadap ambang logika dan menghasilkan salah satu dari dua kondisi, yaitu HIGH atau LOW. Pada ESP32, ambang tersebut berada di kisaran 0,25 dan 0,75 dari tegangan catu (nilai pendekatan dari datasheet), dan tegangan di antara kedua ambang tidak memberikan informasi tambahan. Sementara itu, pada analog input ADC mengukur besarnya tegangan lalu mengonversinya menjadi nilai numerik. ADC 12-bit pada ESP32 membagi rentang tegangan sekitar 0–3,3 V menjadi $2^{12} = 4096$ tingkat (0 hingga 4095), dengan resolusi kira-kira $3{,}3/4095 \approx 0{,}81$ mV per tingkat. Jika salah satu LED digantikan oleh LDR yang dirangkai sebagai pembagi tegangan bersama resistor 10 kΩ, tegangan keluaran akan berubah secara kontinu mengikuti nilai hambatan LDR yang bergantung pada intensitas cahaya. ADC kemudian mengubah tegangan tersebut menjadi salah satu dari 4096 nilai, sehingga sistem dapat membedakan banyak tingkat kecerahan, tidak sekadar terang dan gelap. Satu hal yang perlu diperhatikan, ADC pada ESP32 tidak sepenuhnya linear di dekat batas bawah maupun batas atas rentang pengukurannya.

4 Kesimpulan

Pelaksanaan Praktikum Modul 1 berhasil diselesaikan hingga Percobaan 3. Program Blink dapat dijalankan pada STM32 Blackpill menggunakan framework Arduino melalui ST-Link dengan LED PC13 yang bersifat aktif LOW, serta pada ESP32 menggunakan framework ESP-IDF dengan LED pada GPIO 2. Hasil kedua percobaan sejalan dengan teori, di mana periode kedipan 1000 ms berasal dari dua penundaan 500 ms. Kegagalan upload melalui ST-Link terbukti disebabkan oleh koneksi fisik dan bukan oleh program, sedangkan pada ESP-IDF project harus

diletakkan pada path tanpa spasi dan file program harus menggunakan ekstensi `.c`.

Pada Percobaan 3, hasil pembacaan ketika tombol tidak ditekan telah sesuai dengan teori, yaitu HIGH untuk pull-up dan LOW untuk pull-down. Percobaan ini juga menunjukkan bahwa kesalahan pemasangan kabel dapat menimbulkan pembacaan keliru yang konsisten, seperti GPIO 32 yang selalu terdeteksi ditekan. Berdasarkan teori, rangkaian pull-up bekerja dengan logika aktif LOW, sedangkan pull-down dengan logika aktif HIGH. Untuk kondisi yang berhasil didokumentasikan, resistor internal ESP32 (sekitar 45 kΩ) memberikan hasil pembacaan yang sama dengan resistor eksternal 10 kΩ; perbedaannya terletak pada kekuatan tarikan dan kebutuhan komponen tambahan.

Percobaan 4 dan 5 tidak dikerjakan karena keterbatasan waktu praktikum, sehingga belum dapat ditarik kesimpulan dari pengujian langsung mengenai pengaruh nilai debounce maupun pembacaan ADC. Pada Tugas Modul, simulasi Wokwi membuktikan bahwa metode scanning baris-kolom dengan pull-up internal mampu menghubungkan kesembilan tombol dengan kesembilan LED secara tepat, didukung logika debouncing per tombol dengan jendela kestabilan 30 ms. Meski demikian, efek debouncing terhadap pantulan kontak belum dapat diamati karena simulator tidak memodelkan fenomena tersebut, dan rancangan ini juga belum diuji pada perangkat keras sesungguhnya.

5 Lampiran

5.1 Dokumentasi saat praktikum

Foto dokumentasi kegiatan praktikum tidak tersedia. Bukti pelaksanaan percobaan berupa tangkapan layar pada bagian Langkah-Langkah Percobaan dan Hasil Tugas Modul.